

Titulación: Grado en Ingeniería Informática, Ingeniería en
Sistemas de Información e InfoAde
Curso: 2025-2026. Convocatoria Ordinaria de Mayo
Asignatura: Bases de Datos Avanzadas – Laboratorio

Practica 1: Arquitectura PostgreSQL y almacenamiento físico

ALUMNO 1:

Nombre y Apellidos: _____

DNI: _____

ALUMNO 2:

Nombre y Apellidos: Daniel del Nogal Buchanan

DNI: _____

Fecha: 03/02/2026

Profesor Responsable: _____

Mediante la entrega de este fichero los alumnos aseguran que cumplen con la normativa de autoría de trabajos de la Universidad de Alcalá, y declaran éste como un trabajo original y propio.

En caso de ser detectada copia, se calificará la asignatura como Suspensa – Cero.

Es obligatorio proporcionar una explicación a lo que está ocurriendo en PostgreSQL cuando así se indica en la cuestión. No solo vale poner un pantallazo. La ausencia de una explicación hará que sea invalidada esa cuestión.

Plazos

Trabajo de Laboratorio: semana 2 febrero, 9 febrero, 16 febrero, 23 de febrero y 2 marzo.

Entrega de práctica: día 8 de marzo. Aula Virtual

Documento a entregar: **un fichero con formato ZIP con las respuestas a las cuestiones planteadas en este fichero pdf (prohibido modificar el fichero o entregar otro documento diferente con respuestas y/o imágenes), así como los ficheros de log de postgresql relacionados con la resolución de la práctica y el fichero con el código de generación de los datos de la cuestión 1. El fichero se deberá llamar: DNIdelosAlumnos_PL1.zip**

AMBOS ALUMNOS DEBEN ENTREGAR EL FICHERO EN LA PLATAFORMA.

Introducción

En esta primera práctica se introduce el sistema gestor de bases de datos **PostgreSQL (18.1 la última)**. Está compuesto básicamente de un motor servidor y de una serie de clientes que acceden al servidor y de otras herramientas externas. En esta primera práctica se entrará a fondo en la arquitectura de PostgreSQL, sobre todo en el almacenamiento físico de los datos y del acceso a los mismos. Antes de comenzar es obligatorio configurar lo que se comenta en la cuestión. **Hay que resolver la práctica con consultas SQL.**

Cuestión 0. Configurar el fichero de Error Reporting and Logging de PostgreSQL para que aparezcan recogidas las sentencias SQL DDL (Lenguaje de Definición de Datos) + DML (Lenguaje de Manipulación de Datos) generadas en dicho fichero. No se pide activar todas las sentencias. No activar la duración de la consulta. También se debe de configurar el log para que en el comienzo de la línea de registro de la información del log (“line prefix”) aparezcan vuestros DNI’s. ¿Cómo se ha realizado la configuración?

La configuración se ha realizado modificando el archivo de configuración de PostgreSQL (postgresql.conf), se localizaron y editaron: log_statement a 'mod', el cual se le indica a PostgreSQL que registre todas las sentencias de DDL, log_line_prefix a 'DNI DNI %m' para registrar los DNI y la fecha en la que realizaron modificaciones y por ultimo se verifico log_duration a off para no registrar el tiempo de ejecución de cada consulta

Organización de Archivos en PostgreSQL

Cuestión 1. Crear una nueva Base de Datos que se llame **PL1**. Después crear una tabla **estudiantes** con los siguientes campos:

- estudiante_id: que debe ser un serial empezando por 1.
- nombre: guarda el nombre del estudiante.
- codigo_carrera: guarda el código de carrera del alumno.
- edad: guarda la edad del estudiante.
- indice: guarda el número de créditos superados por el alumno.

Crear un programa que permita generar 30 millones de registros en un fichero de texto que pueda ser cargado posteriormente en la tabla (preferiblemente en Python) con las siguientes propiedades para los siguientes campos, cuyos valores se deben **generar aleatoriamente**.

- Código de carrera: deben ser valores aleatorios entre 0 y 100.
- edad: deben ser valores aleatorios entre 18 y 40 años (incluidos).
- índice: deben ser valores aleatorios entre 0 y 10000

Cargar los datos en la tabla y localizar los ficheros relacionados con la tabla. ¿cómo se localizan? ¿Cuánto ocupan? ¿por qué?

Los ficheros se localizan identificadores numéricos (filenodes). Para localizarlos, se utiliza la fun

El tamaño de 2597 MB se debe a que PostgreSQL no solo guarda los datos puros, sino que añ

Cuestión 2. Calcular teóricamente el tamaño en bloques que ocupa la relación **estudiantes** tal y como se realiza en clase de teoría. ¿Concuerda con el tamaño en bloques que nos proporciona PostgreSQL? ¿Cuál es el factor de bloque medio real de la tabla **estudiantes**? ¿Por qué? Realizar una consulta SQL que obtenga ese valor y comparar con el factor de bloque teórico.

No, no concuerda el tamaño real con el calculado teóricamente. El tamaño real en disco que reporta PostgreSQL es mayor que el calculado teóricamente. El calculo teórico da 132 tuplas por bloque, es decir 227.273 bloques. Y el calculo real da 2.597 MB del archivo que serán 332.416 bloques al dividir entre 8KB. Si en el cálculo teórico obtuvimos 132 tuplas por bloque, el factor de bloque medio real será inferior, situándose aproximadamente en unas 115 a 120 tuplas por bloque. Esto implica que, en lugar de los 227.273 bloques teóricos, la tabla real ocupará cerca de 250.000 a 260.000 bloques.

Esto se debe a cómo PostgreSQL gestiona la memoria física y el alineamiento, aspectos que el cálculo teórico simplificado suele omitir. Alineamiento (Padding) Esta es la causa principal. PostgreSQL alinea los datos en memoria en múltiplos de 8 bytes, en la teoría la tupla suma 57,6 bytes, pero PostgreSQL no puede escribir 57,6 bytes y empezar la siguiente. Redondea hacia arriba al siguiente múltiplo de 8, por lo que cada tupla ocupa realmente 64. FillFactor, por defecto, PostgreSQL puede reservar espacio libre en las páginas para futuras actualizaciones. El factor bloque real es de 120 y el teórico es de 132.

Cuestión 3. Realizar una consulta que muestre los estudiantes que tengan un índice de 500. ¿Cuántas tuplas se obtienen y cuántos bloques se leen del disco por Postgres? ¿Por qué? Obtener las estadísticas de la tabla y sus campos, para comparar con los resultados obtenidos al aplicar el método visto en teoría. (usar funciones de “The Cumulative Statistics System”)

al realizar la consulta se obtienen 2.974 tuplas y 250.048 bloques físicos del disco. Esto ocurre p
El tamaño real de la tabla es de 250.048 bloques, Comparado con el cálculo teórico de la Cuest

Cuestión 4. Volver a realizar la consulta de la cuestión 3 de nuevo. ¿Cuántas tuplas se obtienen y cuántos bloques se leen del disco por Postgres ahora? ¿Por qué? Comparar con la cuestión anterior

Al repetir la consulta, se obtienen las mismas 2.974 tuplas que en la cuestión anterior, ya que los datos no han variado. Sin embargo, a diferencia de lo que predice la teoría ideal se han producido 249.484 lecturas físicas desde el disco y solo 564 aciertos en memoria. Esto ocurre porque el tamaño de la tabla supera ampliamente la memoria asignada a los shared_buffers (por defecto 128 MB), lo que impide que la tabla completa se mantenga en RAM. PostgreSQL se ve obligado a desalojar los bloques antiguos para leer los nuevos, provocando que en la segunda consulta tenga que volver a realizar un Sequential Scan físico casi completo, con un coste de E/S prácticamente idéntico al de la primera vez .

Cuestión 5. Crear una tabla **estudiantes2** cuyas tuplas estén ordenadas físicamente por el campo índice de menor a mayor y que tenga la misma información. Cargar el mismo fichero de datos creado en la cuestión1. Indicar el proceso de generación de dicha tabla ordenada. ¿Cuántos bloques ocupa la tabla ahora? ¿Hay algún cambio? ¿Por qué?

Para realizar la cuestion se ha creado una nueva tabla con el siguiente codigo; CREATE TABLE estudiantes2 AS
SELECT * FROM estudiantes
ORDER BY indice ASC;
para crear una nueva tabla ordenada con indice. Según los datos obtenidos de la vista del sistema pg_class tras ejecutar el ANALYZE, la tabla estudiantes2 ocupa exactamente 250.048 bloques. No hay ningún cambio en el tamaño físico. Al comparar con la tabla original estudiantes, observamos que ambas ocupan el mismo espacio en disco. El tamaño se mantiene idéntico por las siguientes razones. Mismo volumen de información. Ambas tablas contienen exactamente las mismas 30.000.000 de tuplas.

El resultado es:

Tabla estudiantes (Desordenada): 250.048 bloques.

Tabla estudiantes2 (Ordenada): 250.048 bloques.

Conclusión: Ordenar físicamente la tabla no altera su tamaño total.

Cuestión 6. Repetir la cuestión 3 sobre la tabla **estudiantes2** y comparar los resultados obtenidos indicando las conclusiones obtenidas. Relacionarlo con lo visto en teoría.

Se obtienen 2.974 tuplas, el mismo resultado que en la Cuestión 3, ya que la lógica de la consu

Cuestión 7. Borrar 5000000 tuplas de la tabla **estudiantes** de manera aleatoria usando el valor del campo estudiante_id. ¿Qué es lo que ocurre físicamente en la base de datos? ¿Se observa algún cambio en el tamaño de la tabla y estructuras asociadas a ella? ¿Por qué? Adjuntar el código de borrado.

El código utilizado de borrado es DELETE FROM estudiantes WHERE estudiante_id % 6 = 0;
Físicamente, las tuplas no se borran del disco.

PostgreSQL utiliza un sistema llamado MVCC. Cuando se ejecuta un DELETE, el motor no elimina los bytes de la fila en el bloque físico. En su lugar, actualiza la cabecera de la tupla marcándola como eliminada. A partir de ese momento, la tupla se considera una "tupla muerta". Posteriormente, un proceso en segundo plano llamado VACUUM recicla ese espacio interno para futuras inserciones o actualizaciones, pero el archivo en disco nunca se encoge automáticamente; para reducir su tamaño real sería necesario ejecutar un VACUUM FULL, una operación que sí bloquearía la tabla por completo.

El tamaño no disminuye por el diseño arquitectónico de PostgreSQL, que prioriza el rendimiento y la concurrencia. Borrar físicamente y compactar un archivo de 2.5 GB en tiempo real bloquearía la tabla por completo y consumiría demasiados recursos

Cuestión 8. Insertar un nuevo estudiante en la tabla **estudiantes**. ¿dónde se inserta físicamente la nueva tupla? ¿Por qué? (Mirar apartado "System Columns")

Se inserta casi al final del archivo físico de la tabla, concretamente en el bloque 249.990 (posición 25). Esto se comprueba consultando la columna oculta del sistema ctid, que indica las coordenadas físicas exactas (bloque, tupla).

Aunque en el paso anterior borramos 5 millones de registros y hay huecos al principio de la tabla, PostgreSQL no los reutiliza inmediatamente. Las filas borradas solo se marcan como "muertas". Hasta que un proceso automático en segundo plano (VACUUM) no pasa a limpiarlas y registra ese espacio libre en el Mapa de Espacio Libre (FSM), el motor elige la opción más rápida para no perder tiempo buscando. Añadir la nueva tupla al final del fichero, donde sabe seguro que hay sitio.

Cuestión 9. En la situación anterior, ¿Qué operaciones se pueden aplicar a la base de datos **PL1** para optimizar el rendimiento de esta? Aplicarlas de tal manera que se recupere el mayor espacio posible. Comentar cuál es el resultado final y qué es lo que ocurre físicamente. (Mirar “Routine Database Maintenance Tasks”)

El código utilizado para recuperar el espacio es `VACUUM FULL ANALYZE estudiantes`;

Tras ejecutar la orden, las 5.000.000 de tuplas muertas se han eliminado por completo, mientras Para lograr esto, PostgreSQL bloquea la tabla para evitar lecturas y escrituras concurrentes, cre

Cuestión 10. Crear una nueva tabla denominada **estudiantes3** con los mismos campos que la cuestión 1 y que esté particionada por el campo código de carrera por medio de la función $h = \text{código_carrera} \bmod 20$. Insertar los datos del fichero de datos generado en la cuestión 1. Explicar el proceso seguido y comentar qué es lo que ha ocurrido físicamente en la base de datos. ¿Cuándo será útil el particionamiento? ¿Cuántos bloques ocupa cada una de las particiones? ¿Por qué? Comparar con el número bloques que se obtendría teóricamente utilizando el procedimiento visto en teoría. (Ver comando “CREATE TABLE”)

Se ha creado una tabla "padre" (estudiantes3) que lógicamente actúa como una sola tabla, pero

El particionamiento se utiliza para optimizar consultas y mantenimiento: Partition Pruning: Si se

Cálculo teórico: Sabiendo que el factor de bloque teórico era de 132 registros/bloque resulta q
Cálculo práctico: El factor de bloque real de PostgreSQL es de 120 registros/bloque debido a la

Cuestión 11. Repetir la cuestión 3 sobre la tabla **estudiantes3** y comparar los resultados obtenidos con lo visto anteriormente en las tablas **estudiantes** y **estudiantes2** obteniendo conclusiones sobre el método de partición.

Al repetir la consulta de búsqueda (SELECT count(*) FROM estudiantes3 WHERE indice = 500;) :

Indexación de PostgreSQL

PostgreSQL soporta indexación definida por el usuario para ayudar a acelerar ciertas consultas. Entre otros tipos de índices soporta árboles y hash. En este apartado se va a trabajar sobre ambos tipos de índices, pudiendo observar cómo se organizan internamente y su funcionamiento. (Mirar módulos "pgstattuple" y "pageinspect")

Cuestión 12. Borrar todas las tablas **estudiantes**, **estudiantes2** y **estudiantes3**. Crear una nueva tabla que se llama **estudiantes2** como en la cuestión 5 (ordenado por el campo **índice**) y que tenga cargados todos los datos del fichero de texto generado.

Para llevar al cabo la Cuestión 12 se ha utilizado el siguiente código. Para borrar todas las tablas DROP TABLE IF EXISTS estudiantes, estudiantes2, estudiantes3 CASCADE;

Para crear tabla temporal para la carga masiva
CREATE TABLE estudiantes_temp

Para cargar los datos originales del fichero de texto
\copy y para crear la tabla estudiantes2 ordenada físicamente por el campo índice
CREATE TABLE estudiantes2 AS
SELECT * FROM estudiantes_temp
ORDER BY índice ASC;

Por ultimo se elimina la tabla temporal y actualizar estadísticas
DROP TABLE estudiantes_temp; ANALYZE estudiantes2;

Cuestión 13. Crear un índice de tipo árbol para **estudiante_id**. ¿Dónde se almacena físicamente ese índice? ¿Qué tamaño tiene? ¿Cuántos bloques tiene? ¿Cuántos niveles tiene? ¿Cuántos bloques tiene por nivel? ¿Cuántas tuplas tiene un bloque de cada nivel? Indicar el procedimiento seguido e incluir el código SQL utilizado.

Físicamente, el índice se almacena en el disco bajo la ruta base/16388/17028. Tiene un tamaño

el código utilizado es:

```
CREATE EXTENSION IF NOT EXISTS pageinspect;  
CREATE EXTENSION IF NOT EXISTS pgstattuple;
```

```
CREATE INDEX idx_estid_btree ON estudiantes2 USING btree(estudiante_id);
```

```
SELECT pg_relation_filepath('idx_estid_btree') AS ubicacion_fisica,  
       pg_size_pretty(pg_relation_size('idx_estid_btree')) AS tamaño_legible,  
       relpages AS total_bloques  
FROM pg_class WHERE relname = 'idx_estid_btree';
```

```
SELECT level AS nivel_raiz  
FROM bt_metap('idx_estid_btree');
```

```
SELECT internal_pages AS bloques_internos, leaf_pages AS bloques_hoja  
FROM pgstatindex('idx_estid_btree');
```

Cuestión 14. Determinar el tamaño de bloques y el número de niveles que teóricamente tendría de acuerdo con lo visto en teoría. Comparar los resultados obtenidos teóricamente con los resultados obtenidos en la cuestión 13.

Para la Cuestión 14, hemos calculado que cada entrada del índice ocupa unos 20 bytes, 4 del IN

Cuestión 15. Crear un índice de tipo hash para el campo **estudiante_id**. ¿Dónde se almacena físicamente ese índice? ¿Qué tamaño tiene? ¿Cuántos bloques tiene? ¿Cuántos cajones tiene? ¿Cuántas tuplas tiene de media un cajón? Indicar el procedimiento seguido e incluir el código SQL utilizado.

Para resolver la Cuestión 15, he ejecutado el comando CREATE INDEX idx_estid_hash ON estudi

El código utilizado fue:

```
CREATE INDEX idx_estid_hash ON estudiantes2 USING hash(estudiante_id);
```

```
SELECT  
  pg_relation_filepath('idx_estid_hash') AS ubicacion,  
  pg_size_pretty(pg_relation_size('idx_estid_hash')) AS tamaño,  
  relpages AS bloques_totales  
FROM pg_class WHERE relname = 'idx_estid_hash';
```

```
SELECT magic, version, ntuples, ffactor, bsize, bmsize, bmcount  
FROM hash_metapage('idx_estid_hash');
```

Cuestión 16. Determinar el tamaño de bloques y el número de cajones que teóricamente tendría de acuerdo con lo visto en teoría. Comparar los resultados obtenidos teóricamente con los resultados obtenidos en la cuestión 15.

Para la Cuestión 16, hemos comparado el rendimiento de ambos índices usando EXPLAIN ANA

Cuestión 17. Crear un índice de tipo árbol para el campo **índice**. ¿Dónde se almacena físicamente ese índice? ¿Qué tamaño tiene? ¿Cuántos bloques tiene? ¿Cuántos niveles tiene? ¿Cuántos bloques tiene por nivel? ¿Cuántas tuplas tiene un bloque de cada nivel? Indicar el procedimiento seguido e incluir el código SQL utilizado.

Para resolver esta cuestión, hemos creado un índice de tipo árbol sobre el campo indice media

El código utilizado es:

Para crear el índice de tipo árbol

```
CREATE INDEX idx_indice_btree ON estudiantes2(indice);
```

Para consultar ruta, tamaño y bloques totales

```
SELECT  
    pg_relation_filepath('idx_indice_btree') AS ruta,  
    pg_size_pretty(pg_relation_size('idx_indice_btree')) AS tamaño,  
    relpages AS bloques  
FROM pg_class WHERE relname = 'idx_indice_btree';
```

Y para consultar niveles y bloque raíz

```
SELECT magic, version, root, level FROM bt_metap('idx_indice_btree');
```

Cuestión 18. Determinar el tamaño de bloques y número de niveles que teóricamente tendría de acuerdo con lo visto en teoría. Comparar los resultados obtenidos teóricamente con los resultados obtenidos en la cuestión 17.

Para la Cuestión 18, el tamaño teórico de cada bloque en PostgreSQL es de 8 KB (8192 bytes), l

Cuestión 19. Crear un índice de tipo hash para el campo **índice**. ¿Dónde se almacena físicamente ese índice? ¿Qué tamaño tiene? ¿Cuántos bloques tiene? ¿Cuántos cajones tiene? ¿Cuántas tuplas tiene de media un cajón? Indicar el procedimiento seguido e incluir el código SQL utilizado.

Para la Cuestión 19, hemos creado un índice hash sobre el campo índice que se encuentra alm

El código usado es:

```
DROP INDEX IF EXISTS idx_indice_hash;  
CREATE INDEX idx_indice_hash ON estudiantes2 USING hash(indice);
```

```
SELECT  
    pg_relation_filepath('idx_indice_hash') AS ruta,  
    pg_size_pretty(pg_relation_size('idx_indice_hash')) AS tamaño,  
    relpages AS bloques  
FROM pg_class  
WHERE relname = 'idx_indice_hash';
```

```
SELECT maxbucket + 1 AS nbuckets  
FROM hash_metapage_info(get_raw_page('idx_indice_hash', 0));
```

```
SELECT (30000000 / (maxbucket + 1)) AS media_tuplas_cajon  
FROM hash_metapage_info(get_raw_page('idx_indice_hash', 0));
```

Cuestión 20. Determinar el tamaño de bloques y número de cajones que teóricamente tendría de acuerdo con lo visto en teoría. Comparar los resultados obtenidos teóricamente con los resultados obtenidos en la cuestión 17.

El tamaño de bloque teórico en PostgreSQL es de 8 KB, valor que coincide con la realidad del s

Cuestión 21. ¿Qué conclusiones se puede obtener de la gestión y organización de PostgreSQL sobre los dos índices árbol y hash que se han creado y han sido analizados? ¿Por qué? Comparar con lo visto en teoría.

Como conclusión del análisis, la gestión de PostgreSQL demuestra que el índice B-Tree es mucl

Monitorización de la actividad de la base de datos

En este último apartado se mostrará el acceso a los datos con una serie de consultas sobre la tabla original. En este apartado se pretende mostrar cómo es el acceso a los datos para diferentes tipos de consultas.

PostgreSQL suministra varias vistas estadísticas que se pueden usar para monitorizar los bloques leídos (tipo statio de la sección The Cumulative Statistics System) de cada una de las estructuras creadas en la base de datos. En este apartado se deben usar esas vistas y está prohibido el uso de otro comando para este fin (Table 28.2).

Para ello, borrar todas las tablas creadas y volver a crear la tabla **estudiantes** como en la cuestión 1. Cargar los datos que se encuentran originalmente en el fichero generado en la cuestión 1.

Cuestión 22. Crear un índice primario tipo árbol sobre el campo **índice**, y otro sobre **código_carrera**. También crear un índice hash sobre el campo **estudiante_id** y otro sobre **índice**. ¿Cuál ha sido el proceso seguido para crear cada tipo de índice? Incluir el código SQL utilizado para ello.

El proceso consiste en utilizar la sentencia CREATE INDEX definiendo el nombre del índice, la tabla.

Para los índices de tipo árbol, se especifica USING btree

Para los índices de tipo hash, se especifica USING hash.

-- Índices tipo Árbol

```
CREATE INDEX idx_tree_indice ON estudiantes USING btree (indice);
```

```
CREATE INDEX idx_tree_carrera ON estudiantes USING btree (codigo_carrera);
```

-- Índices tipo Hash

```
CREATE INDEX idx_hash_est_id ON estudiantes USING hash (estudiante_id);
```

```
CREATE INDEX idx_hash_indice ON estudiantes USING hash (indice);
```

Cuestión 23. Para cada una de las consultas que se muestran a continuación, ¿Qué información se puede obtener de los datos monitorizados por la base de datos al realizar la consulta? Comentar cómo se ha realizado la resolución de la consulta. ¿Cuántas tuplas se han obtenido? ¿Cuántos bloques se han leído de cada estructura? ¿Por qué? Comparar con lo visto en teoría. **Importante, reinicializar los datos recolectados de la actividad de la base de datos antes de lanzar cada consulta.** Se recuerda que solo se pueden usar vistas sobre las estadísticas de la base de datos (**usar funciones de “The Cumulative Statistics System”**).

1. Mostrar la información de las tuplas con código de carrera igual 50.

Se puede obtener de una consulta: Los bloques leídos físicamente del disco (blks_read), los bloques leídos por el índice. La consulta se resolvió utilizando primero el índice del campo carrera, leyendo solo 254 bloques. Se han leído 254 bloques del índice. Se han leído 174.134 bloques de la tabla principal estudiantes. Se han obtenido 296738 tuplas.

Cálculo teórico:

Para código_carrera = 50, con 101 valores posibles 0-100, se obtienen teóricamente 30.000.000

2. Mostrar la información de la tupla con estudiante_id igual a 80000.

Para resolver esta consulta de búsqueda exacta, el sistema utilizó el índice Hash (idx_hash_est_i

Calculo teórico:

Debido a que estudiante_id es un campo de tipo serial, se obtendrá exactamente 1 tupla. Teóri

3. Mostrar los datos de los estudiantes que tienen un índice entre 100 y 200.

El optimizador descartó automáticamente el índice Hash, ya que su estructura matemática no p

Calculo teórico:

El rango incluye 101 valores posibles $200 - 100 + 1$ de 10.001 valores (de 0 a 10.000). La cantidad

4. Contar el número de estudiantes que tienen una edad de 20 años.

Para resolver esta consulta, el gestor de la base de datos tuvo que realizar un escaneo secuenci

Consulta teórica:

La consulta de edad = 20, al carecer de índice, obliga a un Sequential Scan que lee los 250.048

5. Mostrar el número total de estudiantes que tienen el mismo índice.

Para resolver esta consulta de agrupación sobre toda la tabla, el gestor de la base de datos ha :

Consulta teórica: Para obtener el número de estudiantes teóricamente por cada uno de los 10.0

6. Obtener la media de edad de los estudiantes que tienen la misma carrera.

Para resolver esta consulta que calcula la media de edad agrupada por carrera (SELECT codigo_

Calculo teórico:

El cálculo del promedio de edad para las 101 carreras posibles exige acceder a los datos de edad

7. Insertar un nuevo estudiante con un índice de 100.

Para resolver esta consulta de inserción de un nuevo estudiante, el gestor de la base de datos c

Calculo teórico:

La inserción de una nueva tupla implica la escritura de 1 bloque físico en la tabla de datos. A es

Cuestión 24. Borrar los índices creados en la cuestión 20. Crear un índice multiclave tipo árbol sobre los campos **código_carrera e índice**. Incluir el código SQL utilizado para ello.

Con estas sentencias primero eliminamos cualquier estructura de acceso secundaria previa para:

```
DROP INDEX IF EXISTS idx_tree_indice;  
DROP INDEX IF EXISTS idx_tree_carrera;  
DROP INDEX IF EXISTS idx_hash_est_id;  
DROP INDEX IF EXISTS idx_hash_indice;  
CREATE INDEX idx_multiclave_carrera_indice  
ON estudiantes USING btree (codigo_carrera, indice);
```

Cuestión 25. Para cada una de las consultas que se muestran a continuación, ¿Qué información se puede obtener de los datos monitorizados por la base de datos al realizar la consulta? Comentar cómo se ha realizado la resolución de la consulta. ¿Cuántos bloques se han leído de cada estructura? ¿Por qué? Importante, reinicializar los datos recolectados de la actividad de la base de datos antes de lanzar cada consulta:

1. Mostrar los estudiantes que tienen un código de carrera de 20 y un índice de 500.

Porque la consulta filtra con un AND utilizando los dos campos exactos por los que se ha creado

2. Mostrar los estudiantes que tienen un código de carrera de 50 o un índice de 900.

Del índice multiclave: Se han leído 471 bloques físicos del disco y 114 desde la caché
De la tabla principal: Se han leído masivamente 174.977 bloques físicos y 3 desde la caché.

Al utilizar el operador OR el gestor no puede hacer una única búsqueda directa hacia una hoja
Una vez obtenida la lista de punteros, tiene que acceder a la tabla para extraer los datos completos

3. Mostrar los estudiantes que tienen un índice de 300.

Del índice multiclave: Se han leído 202 bloques físicos del disco y 108 desde la caché.

De la tabla principal: Se han registrado 0 bloques leídos.

El gestor se enfrenta a un problema de estructura: la consulta filtra únicamente por el campo `ir`. Para encontrar los datos, el motor se ve forzado a explorar las ramas del árbol buscando qué c

4. Mostrar los estudiantes que tienen un código de carrera de 80.

Del índice multiclave: Se han leído 271 bloques físicos del disco y 4 desde la caché.

De la tabla principal: Se ha realizado una lectura masiva de 174.228 bloques físicos y 153 desde

En este caso, la consulta filtra únicamente por el campo `codigo_carrera`. Al ser esta la primera c

Cuestión 26. Crear la tabla **estudiantes3** particionada por el campo **edad** de tal manera que los estudiantes que tengan la misma edad estén juntos en la misma partición. Para cada una de las consultas que se muestran a continuación, ¿Qué información se puede obtener de los datos monitorizados por la base de datos al realizar la consulta? Comentar cómo se ha realizado la resolución de la consulta. ¿Cuántos bloques se han leído de cada estructura? ¿Por qué? Comparar con la teoría. Importante, reinicializar los datos recolectados de la actividad de la base de datos antes de lanzar cada consulta.

1. Mostrar el número de estudiantes con una edad de 25.

De la tabla padre o del resto de particiones: 0 bloques. De la partición hija afectada: Se han esc

2. Mostrar los estudiantes que tienen una edad de 20 o 25 o 30 años.

Al filtrar usando el operador OR o IN sobre la clave de partición (edad), el gestor de la base de datos vuelve a aplicar de forma perfecta la poda de particiones (Partition Pruning). El optimizador identifica que solo necesita acceder a las tres tablas físicas correspondientes a esas edades exactas y descarta automáticamente las otras 83 particiones restantes.

estudiantes3_edad_20: 10.214 bloques físicos leídos y 669 de caché.

estudiantes3_edad_25: 9.937 bloques físicos leídos y 938 de caché.

estudiantes3_edad_30: 9.974 bloques físicos leídos y 882 de caché.

Consultas teóricas: Se acceden a tres particiones físicas de forma independiente. El número total de tuplas es de $3 * 1.304.348 = 3.913.044$ registros, con un coste de lectura de $3 * 9.881 = 29.643$ bloques teóricos

3. Mostrar los estudiantes que tienen una edad entre 25 y 30 años (incluidos).

estudiantes3_edad_25: 9.843 bloques leídos y 1.032 de caché.

estudiantes3_edad_26: 10.221 bloques leídos y 655 de caché.

estudiantes3_edad_27: 10.217 bloques leídos y 657 de caché.

estudiantes3_edad_28: 9.969 bloques leídos y 915 de caché.

estudiantes3_edad_29: 9.991 bloques leídos y 877 de caché.

estudiantes3_edad_30: 9.880 bloques leídos y 976 de caché.

Resto de particiones: 0 bloques leídos.

Al utilizar una condición de rango sobre la clave de partición, el optimizador de PostgreSQL demuestra que Partition Pruning también funciona con intervalos continuos. El motor calcula qué tablas físicas cubren los valores del 25 al 30 y descarta de inmediato cualquier otra tabla de la base de datos. Cálculo teórico: El rango abarca 6 particiones edades 25, 26, 27, 28, 29 y 30. Se recuperan $6 * 1.304.348 = 7.826.088$ tuplas, lo que supone un esfuerzo físico de $6 * 9.881 = 59.286$ bloques teóricos.

4. Mostrar los estudiantes que tienen una edad mayor de 50 años.

Bloques leídos: 0 bloques en total. No se ha registrado ninguna lectura ni en disco ni en caché en ninguna de las particiones.

Filas devueltas: 0 filas.

Este resultado nos revela una característica del conjunto de datos original: no existe ningún estudiante mayor de 50 años en la base de datos. El motor aplicó la poda de particiones y determinó que debía buscar en las tablas desde estudiantes3_edad_51 hasta estudiantes3_edad_100. Sin embargo, al ir a acceder a ellas, comprobó en los metadatos que esos archivos físicos estaban completamente vacíos. Al no haber bloques de datos que leer, el coste ha sido cero.

Cálculo teórico:

Como no existen valores mayores de 50 años, no existen registros que cumplan esta condición. El sistema hará 0 lecturas de bloques y devuelve 0 tuplas, ya que se descartan todas las particiones al comprobar que el rango de búsqueda está fuera de los límites de los datos almacenados.

Cuestión 27. A la vista de los resultados obtenidos de este apartado, comentar las conclusiones que se pueden obtener del acceso de PostgreSQL a los datos almacenados en disco.

En conclusión, el mayor problema de rendimiento en PostgreSQL es tener que leer del disco fís

Bibliografía (PostgreSQL 18.1)

- Capítulo 1: Getting Started.
- Capítulo 5: 5.6 System Columns.
- Capítulo 5: 5.12 Table Partitioning.
- Capítulo 11: Indexes.
- Capítulo 19: Server Configuration.
- Capítulo 24: Routine Database Maintenance Tasks.
- Capítulo 27: Monitoring Database Activity. The statistics Collector
- Capítulo 27.6: Monitoring Disk Usage. Determining Disk Usage
- Capítulo 52: System Catalogs
- Capítulo 65.1: B-Tree Indexes
- Capítulo 65.6: Hash Indexes
- Capítulo 67: Database Physical Storage
- Capítulo VI.II: PostgreSQL Client Applications.
- Capítulo VI.III: PostgreSQL Server Applications.
- Apéndice F: Additional Supplied Modules. Pageinspect, pgstatutuple
- Apéndice G: Additional Supplied Programs. Oid2name